

Poli Insight Database and System Model Design

Status: recommended pre-implementation design

Target stack: Python 3.12, Streamlit, SQLAlchemy, Alembic, SQLite for development, PostgreSQL for production

Primary qualities: reproducibility, auditability, security, portability, and algorithm extensibility

1. Recommended architecture and major design decisions

1.1 Architectural shape

Use four layers, matching the direction already present in the repository:

1. **Domain layer** owns lifecycle rules and invariants for sessions, participants, submissions, validations, and runs.
2. **Application layer** coordinates transactions and use cases through repository and unit-of-work interfaces.
3. **Infrastructure layer** maps domain objects to SQLAlchemy models and invokes algorithm adapters, artifact storage, email, and AI providers.
4. **Presentation layer** contains Streamlit pages and forms but does not enforce the only copy of a business rule.

Keep the relational database as the system of record. Treat algorithm execution and AI report generation as replaceable services behind application ports. PyDecision is one implementation provider, not part of the database's conceptual model.

The central lineage should be:

```
scenario snapshot + frozen session configuration + immutable submissions
-> versioned validations
-> immutable processing run input set
-> structured processing outputs
-> versioned analysis runs
-> audience-specific report outputs
```

1.2 Rename the **ScenarioSession** concept

Use **ScenarioSnapshot** for the requested **ScenarioSession** concept. **ScenarioSession** is too easily confused with the central **Session**. A snapshot is an immutable, content-addressed package of every input needed to reconstruct the decision problem.

A scenario snapshot must include more than the current configuration JSON:

- the scenario, criteria, data-source, preprocessing, session-rule, and UI documents;
- every referenced source file, including CSV content;
- the exact custom-function source used during import, retained as evidence;
- a canonical, materialized alternative-by-criterion decision matrix;

- criteria, alternatives, scale definitions, and provenance metadata;
- a manifest and SHA-256 digest for every component plus a root digest.

The calculation pipeline should use the materialized decision matrix, not rerun arbitrary snapshotted Python during normal processing. This both improves reproducibility and avoids treating stored source code as trusted executable code. Re-importing a new or edited scenario produces a new snapshot; existing snapshots are never updated in place.

1.3 Use immutable versions at every calculation boundary

The mutable **sessions** row is an operational shell. Its calculation-relevant settings belong in immutable **session_configuration_versions** rows. A draft can create new configuration versions. Opening a session pins one active version; a submission and every run reference that exact version.

After the first submitted response, changes to any of the following require a new session (recommended) or a formally forked configuration with explicit submission compatibility rules:

- scenario snapshot;
- criteria, alternatives, scale, or response format;
- weighting, aggregation, or ranking algorithms and parameters;
- stakeholder group definitions or allocations.

Administrative scheduling changes, pausing, closing, and access revocation may remain mutable, but every change must be audited.

1.4 Separate discoverability from admission control

Do not model **public**, **private**, and **unlisted** as mutually exclusive access levels. They mix two independent questions:

- **Discoverability:** does the session appear in public listings?
- **Admission:** who is allowed to enroll or enter?

Recommended fields are:

- **discoverability:** **listed** or **unlisted**;
- **enrollment_mode:** **open** or **invitation_only**;
- **access_code_mode:** **none**, **shared_session_code**, or **per_invitation_code**.

The ordinary product presets then become:

Product label	Discoverability	Enrollment mode	Typical code mode
Public	listed	open	none or shared_session_code
Unlisted link	unlisted	open	none or shared_session_code
Private	unlisted	invitation_only	none or per_invitation_code
Visible but controlled	listed	invitation_only	optional

Thus, **unlisted is meaningful as a discoverability state, not as a third authorization state**. If the UI only needs two presets, expose Public and Private while retaining the orthogonal fields internally.

1.5 Normalize core facts; use JSON for versioned documents and artifacts

Use relational columns and foreign keys for identity, ownership, state, ordering, inclusion, and commonly queried outputs. Use JSON for algorithm-specific parameters, raw response envelopes, diagnostics, environment manifests, and general artifacts whose shape varies by algorithm.

Do not make one large JSON document the only representation of submissions or results. Important facts such as criterion weights and alternative rankings need structured rows so that comparisons, reports, constraints, and indexes remain reliable.

JSON documents must carry a `schema_version` and be canonicalized before hashing. Persist the digest beside the content.

1.6 Preserve attempts and outputs; never overwrite history

- A submitted response is immutable.
- Resubmission creates a new attempt and supersedes the former effective submission.
- Validation is repeatable and versioned; it does not edit a submission.
- A processing or analysis run is append-only after completion or failure.
- Publishing a report creates a publication record; it does not change the underlying run.
- Withdrawal, invalidation, revocation, and redaction are explicit state changes or overlay records, not hard deletion of evidence.

1.7 Stable stakeholder-group voting power

Participant counts must not change group power. Compute in two stages:

1. Aggregate valid participant weights **within each group** using the configured within-group method. Equal participant influence should be the default.
2. Combine group-level weight vectors using the frozen group allocations.

For criterion `c`:

```
aggregate_weight[c] = sum(group_allocation[g] * group_weight[g, c])
```

The run must record what happened when an active group has no valid submissions. Recommended default: fail readiness validation and require moderator action. Silently reallocating an absent group's power changes the decision rule and should occur only under an explicit `missing_group_policy`.

1.8 Portable data choices

- Store IDs as application-generated UUIDs represented consistently (for example `CHAR(36)` through a SQLAlchemy type decorator), avoiding backend-generated identity assumptions.
- Store timestamps in UTC. Require timezone-aware Python values; normalize on write because SQLite's timezone behavior is limited.
- Use `Numeric(p, s)` or integer basis points for voting allocations and published numeric weights, not binary `Float` where exact sums matter.

- Use SQLAlchemy **JSON** for JSON documents and **LargeBinary** for inline artifacts. Do not depend on PostgreSQL-only JSONB operations in domain logic.
- Store enum values as lowercase strings with portable **CHECK** constraints. Avoid database-native enum types so adding a value remains an ordinary migration.
- Name every foreign key, unique constraint, check constraint, and index for predictable Alembic migrations.
- Enable SQLite foreign keys on every connection. Treat PostgreSQL as the concurrency and production-integrity target.

2. Tables/entities and their purpose

The tables are grouped by bounded responsibility. Tables marked **immutable** must not be updated after finalization except for tightly controlled redaction metadata.

2.1 Identity and administration

Entity	Purpose
users	Optional authenticated administrator/moderator identity. Participant enrollment does not require a user account.
session_role_assignments	Assigns owner, moderator, analyst, auditor, or report-approver roles to users for a session.

If authentication is delegated to an external identity provider, **users** stores the provider and immutable subject identifier, not passwords.

2.2 Scenario definition and immutable inputs

Entity	Purpose
scenario_definitions	Stable catalog identity for a scenario across imported versions.
scenario_snapshots	Immutable. One reproducible imported version with root hash, manifest, provenance, and status. This implements the requested ScenarioSession concept.
scenario_snapshot_files	Immutable. Every source/config/data/code file or immutable object reference, with logical path, role, bytes, media type, size, and digest.
scenario_criteria	Immutable. Queryable criteria belonging to a snapshot, including direction, data type, unit, hierarchy, and ordering.
scenario_alternatives	Immutable. Queryable alternatives belonging to a snapshot.
scenario_scales	Immutable. Named scale definitions supported by a snapshot.
scenario_scale_values	Immutable. Ordered crisp and/or fuzzy values for a scale.
scenario_matrix_values	Immutable. Canonical materialized alternative-by-criterion input values used for ranking.

Keeping normalized children as well as the complete snapshot manifest is intentional: the manifest proves exactly what was imported, while normalized rows support foreign keys and reliable queries.

2.3 Sessions and frozen configuration

Entity	Purpose
<code>sessions</code>	Operational lifecycle, display, schedule, discoverability, admission controls, and pointer to the active frozen configuration.
<code>session_configuration_versions</code>	Immutable after activation. Calculation and submission rules for one version. Stores configuration JSON plus a hash.
<code>session_stakeholder_groups</code>	Snapshot-specific group name, description, allocation, display order, active flag, and within-group aggregation rule.
<code>algorithm_implementations</code>	Registry of weighting, aggregation, ranking, validation, and analysis implementations independent of a particular session.
<code>session_algorithm_configs</code>	Immutable. Algorithm role, implementation, parameter JSON, parameter schema version, order, and configuration hash for a session configuration version.
<code>response_question_definitions</code>	Immutable. The exact expected questions/targets for a configuration: criterion pair, direct criterion rating, or direct alternative rating/ranking.

`response_question_definitions` removes ambiguity from answer keys and makes completeness verifiable without interpreting arbitrary JSON.

2.4 Access, invitations, and participation

Entity	Purpose
<code>session_invitations</code>	One invitation, its intended group/identity binding, one-time token digest, expiry, redemption, revocation, and send metadata.
<code>session_access_codes</code>	Hashed shared or invitation-specific codes, validity interval, usage limits, revocation, and key-derivation metadata.
<code>access_attempts</code>	Security log for accepted/rejected invite or code attempts, rate-limiting keys, and reason codes. Keep retention limited.
<code>participants</code>	One person's or anonymous respondent's enrollment in exactly one session.
<code>participant_identities</code>	Optional one-to-one PII record separated from analytical data and encrypted where feasible.
<code>participant_access_grants</code>	Revocable, expiring digest of a participant login/resume token. This is separate from the one-time invitation token.

An invitation is a claim that may be redeemed once. A participant is the resulting enrollment. An access grant is a renewable/revocable credential for returning to that enrollment.

2.5 Submissions and answers

Entity	Purpose
<code>submissions</code>	Immutable after submission. One draft or submitted attempt, pinned to participant, group, scenario snapshot, and session configuration version.
<code>submission_answers</code>	One raw answer per expected question, including typed target references, raw value, timing, and final answer hash.
<code>submission_comments</code>	Participant comments with scope, audience, optional criterion/alternative link, and moderation/redaction state.

On final submission, also store a canonical `answer_manifest_json` and `answers_hash` on `submissions`. Answer rows remain the queryable source; the manifest is the immutable signed-off envelope. Normalized answers are validator outputs, not participant-authored facts.

2.6 Validation and generated participant weights

Entity	Purpose
<code>submission_validations</code>	Immutable. One validation attempt tied to exact answer/config/validator hashes, with overall status and metrics.
<code>validation_messages</code>	Structured error/warning/info messages with stable code, field/question reference, parameters, and safe display message.
<code>validation_normalized_answers</code>	Versioned normalized value per submitted question.
<code>participant_criterion_weights</code>	Participant-level crisp or fuzzy criterion weights generated by a validation.

A validity warning is distinct from invalidity. A processing run selects a specific validation record, never merely "the latest validation" without preserving its ID.

2.7 Processing and ranking output

Entity	Purpose
<code>processing_runs</code>	Immutable after terminal state. One reproducible calculation attempt for a session and frozen configuration.
<code>processing_run_algorithms</code>	Immutable. Exact algorithm implementations, parameters, package/library versions, and role used by a run.
<code>processing_run_submissions</code>	Complete candidate roster with included/excluded decision, selected validation, frozen group, and reason.

Entity	Purpose
<code>group_criterion_weights</code>	Structured criterion weights produced for each stakeholder group.
<code>aggregate_criterion_weights</code>	Structured cross-group criterion weights.
<code>alternative_rankings</code>	Structured per-alternative scores, ranks, ties, and optional group scope.
<code>run_artifacts</code>	Versioned matrices, normalization traces, distance calculations, logs, diagnostics, and clean result documents.

`run_artifacts` should include a required `structured_result` artifact whose documented schema is stable enough for downstream analyses and AI report generation.

2.8 Analysis, AI, reports, and publication

Entity	Purpose
<code>analysis_runs</code>	Immutable after terminal state. Sensitivity, robustness, stakeholder comparison, participant-impact, comment-summary, or other analysis against one processing run.
<code>analysis_run_subjects</code>	Optional groups, participants, criteria, or alternatives included in an analysis.
<code>analysis_artifacts</code>	Structured inputs, cases, findings, charts/data, diagnostics, and AI-ready analysis bundle.
<code>report_runs</code>	One deterministic-template or AI generation attempt, with audience, exact input artifacts, prompt/template version, model metadata, and status.
<code>report_outputs</code>	Generated report content and structured claims, redaction/approval status, content hash, and audience scope.
<code>publications</code>	Records which approved processing result/report is visible, its URL key, publication time, version, and withdrawal state.

Participant reports must be authorized by `participant_id`; stakeholder reports by `session_stakeholder_group_id`; moderator reports by session role. A report for one participant must never contain another participant's raw answers or identity.

2.9 Audit and tamper evidence

Entity	Purpose
<code>audit_events</code>	Append-only meaningful actions with actor, target, reason, request/correlation ID, before/after or patch, and timestamp.
<code>audit_event_links</code>	Links one event to additional related entities when a single action spans a transaction.

Entity	Purpose
<code>audit_chain_checkpoints</code>	Optional signed or externally anchored digest checkpoints for stronger tamper evidence.

The audit trail must not contain invitation tokens, access codes, password-equivalent values, unnecessary PII, or unredacted secrets.

3. Relationships, cardinality, and ownership

3.1 High-level relationship map

```

ScenarioDefinition 1 --- * ScenarioSnapshot
ScenarioSnapshot  1 --- * SnapshotFile / Criterion / Alternative / Scale /
MatrixValue

ScenarioSnapshot   1 --- * Session
Session            1 --- * SessionConfigurationVersion
SessionConfigurationVersion 1 --- * SessionStakeholderGroup
SessionConfigurationVersion 1 --- * SessionAlgorithmConfig
SessionConfigurationVersion 1 --- * ResponseQuestionDefinition

Session            1 --- * Invitation / AccessCode / Participant
Invitation         0..1 --- 0..1 Participant
Participant        1 --- * Submission
Submission         1 --- * SubmissionAnswer / SubmissionComment /
SubmissionValidation
SubmissionValidation 1 --- * NormalizedAnswer / ParticipantCriterionWeight

Session            1 --- * ProcessingRun
ProcessingRun       * --- * SubmissionValidation (through
ProcessingRunSubmission)
ProcessingRun       1 --- * GroupCriterionWeight / AggregateCriterionWeight
/ AlternativeRanking / RunArtifact

ProcessingRun       1 --- * AnalysisRun
AnalysisRun         1 --- * AnalysisArtifact
ProcessingRun or AnalysisRun 1 --- * ReportRun
ReportRun           1 --- * ReportOutput
ReportOutput        1 --- * PublicationVersion

```

3.2 Ownership and deletion rules

- `scenario_definitions` own snapshots, but snapshots referenced by a session are **RESTRICT**-delete and normally archived, never deleted.
- A session owns configuration versions, groups, questions, invitations, codes, participants, and runs. Once any submission exists, hard deletion of the session is prohibited.
- A configuration version owns its groups, questions, and session algorithm configurations.
- A participant belongs to one session and one group in the configuration used for participation. Group changes are allowed only before a draft/submission exists; otherwise create a replacement participant

enrollment or explicit membership history.

- A submission belongs to one participant and snapshots `session_id`, `configuration_version_id`, `scenario_snapshot_id`, and `stakeholder_group_id`. Composite foreign keys must prove these all belong to the same session/configuration.
- Each submitted attempt may have many validations. Validations own their messages, normalized answers, and participant weights.
- A processing run belongs to one session/configuration/snapshot and has many run-submission decisions. Every selected validation must belong to the listed submission.
- An analysis run belongs to exactly one processing run. Analyses may reference other runs for comparison through an optional `analysis_comparison_runs` join table, while keeping one primary run.
- Report runs consume explicitly listed artifact IDs. Report outputs and publications never own or mutate calculation records.
- Audit events are retained independently of the target entity's current state. Generic target identifiers are intentionally not cascading foreign keys.

3.3 Cross-session integrity

Surrogate UUID primary keys alone do not prevent accidental cross-session links. Add composite alternate keys and foreign keys for important boundaries, for example:

- unique (`session_id`, `configuration_version_id`) on configuration versions;
- unique (`configuration_version_id`, `stakeholder_group_id`) on groups;
- participant FK (`configuration_version_id`, `stakeholder_group_id`) to its group;
- submission FK (`session_id`, `participant_id`) to a participant alternate key;
- submission FK (`configuration_version_id`, `stakeholder_group_id`) to its group;
- processing-run-submission FK (`submission_id`, `validation_id`) to a validation alternate key.

These constraints make a whole class of application bugs impossible in both SQLite and PostgreSQL.

4. Recommended enums and allowed values

Persist the lowercase values shown below. Keep display labels in application code or localization tables.

Enum	Allowed values	Notes
<code>scenario_snapshot_status</code>	<code>validating</code> , <code>ready</code> , <code>invalid</code> , <code>retired</code>	A ready snapshot is immutable.
<code>scenario_file_role</code>	<code>scenario_config</code> , <code>criteria_config</code> , <code>data_source_config</code> , <code>preprocessing_config</code> , <code>session_rules</code> , <code>ui_config</code> , <code>source_data</code> , <code>custom_function</code> , <code>other</code>	Extendable string enum.

Enum	Allowed values	Notes
<code>criterion_direction</code>	<code>cost, benefit</code>	Add <code>target</code> only when target-value semantics are defined.
<code>criterion_data_type</code>	<code>numeric, ordinal, categorical, boolean</code>	Algorithms must declare supported types.
<code>session_status</code>	<code>draft, scheduled, open, paused, closed, cancelled, archived</code>	Do not use <code>processed</code> or <code>published</code> ; multiple runs/publications can exist.
<code>discoverability</code>	<code>listed, unlisted</code>	Not an authorization decision.
<code>enrollment_mode</code>	<code>open, invitation_only</code>	Whether self-enrollment is possible.
<code>access_code_mode</code>	<code>none, shared_session_code, per_invitation_code</code>	Code is an additional factor, not visibility.
<code>stakeholder_selection_mode</code>	<code>self_select, invitation_assigned, moderator_assigned</code>	Private defaults to invitation assigned.
<code>response_format</code>	<code>pairwise, direct_rating, direct_ranking</code>	Do not collapse scoring and ranking.
<code>response_target_type</code>	<code>criterion, alternative</code>	Resolves the current direct-response ambiguity.
<code>question_type</code>	<code>criterion_pair, criterion_rating, alternative_rating, alternative_rank</code>	Exact expected response unit.
<code>algorithm_role</code>	<code>weighting, within_group_aggregation, across_group_aggregation, ranking, validation, analysis</code>	Role is separate from provider/name.
<code>algorithm_provider</code>	free stable key such as <code>pydecision</code> or <code>internal</code>	Use a catalog row rather than a closed enum if third parties are expected.

Enum	Allowed values	Notes
participant_access_status	active, disabled, withdrawn	Separate from progress.
participant_progress_status	invited, enrolled, started, submitted, completed	May be derived; timestamps remain authoritative.
invitation_status	pending, sent, redeemed, expired, revoked, delivery_failed	Expired may be derived from time but is useful for operations.
submission_status	draft, submitted, superseded, withdrawn	Submitted content is immutable.
validation_status	pending, running, valid, valid_with_warnings, invalid, error	error means validation could not complete, not that participant input was invalid.
message_severity	info, warning, error	Pair with stable machine code.
run_status	queued, running, succeeded, failed, cancelled	Used by processing, analysis, and report runs.
run_inclusion_status	included, excluded_invalid, excluded_superseded, excluded_withdrawn, excluded_disabled, excluded_cutoff, excluded_moderator, excluded_error	Store a detail reason too.
missing_group_policy	fail, exclude_and_renormalize, zero_contribution	fail is recommended.
analysis_type	sensitivity, robustness, stakeholder_comparison, participant_impact, comment_summary, custom	Parameters specify the concrete method.

Enum	Allowed values	Notes
artifact_type	input_manifest, decision_matrix, pairwise_matrix, normalized_matrix, weight_vector, ranking_trace, diagnostic, structured_result, analysis_bundle, log, other	General but schema-versioned.
report_audience	stakeholder_group, moderator, participant, public	Scope ID is required where relevant.
report_approval_status	draft, review_required, approved, rejected, withdrawn	AI output should require review before public publication.
actor_type	user, participant, system, ai_service, import_process, api_client	actor_id may be null only for a documented system bootstrap event.
audit_action	stable verbs such as created, updated, opened, closed, invited, redeemed, submitted, validated, included, excluded, processed, analyzed, approved, published, withdrawn, redacted	Store as a constrained vocabulary or registry table.

Do not encode AHP, Fuzzy AHP, TOPSIS, and Fuzzy TOPSIS as permanent database enums. Seed them as `algorithm_implementations` rows:

Stable key	Role	Provider
ahp	weighting	pydecision
fuzzy_ahp	weighting	pydecision
topsis	ranking	pydecision
fuzzy_topsis	ranking	pydecision

The catalog approach permits another library or version while retaining the stable conceptual method and exact implementation metadata.

5. Constraints, uniqueness rules, indexes, and integrity checks

5.1 Global conventions

- Every table has a single UUID primary key unless a small immutable child naturally uses a composite key.

- Every mutable table has `created_at`, `created_by`, `updated_at`, and `updated_by`; immutable rows have `created_at` and creator plus finalization metadata.
- Every temporal range enforces `end_at IS NULL OR end_at > start_at`.
- Every digest is lowercase SHA-256 hex of length 64 and is checked in application/domain validation.
- Every JSON value is a JSON object/array of the documented shape, has a schema version, excludes NaN/Infinity, and is canonicalized for hashing.
- Blank strings are rejected in domain logic; database `CHECK (length(trim(value)) > 0)` may be added for critical keys/names where both backends behave consistently.
- Use `ON DELETE RESTRICT` for evidence and calculation lineage. Use `CASCADE` only for children of a never-finalized draft or tightly owned value rows. Prefer archival after meaningful activity.

5.2 Scenario constraints and indexes

- `scenario_definitions.scenario_key` unique.
- `scenario_snapshots.root_hash` unique, making identical content idempotent.
- Unique `(scenario_definition_id, declared_version, root_hash)`; do not assume declared version alone proves unique content.
- `scenario_snapshot_files`: unique `(scenario_snapshot_id, logical_path)` and index `(content_hash)`.
- Require exactly one of `inline_bytes` or `immutable_object_uri`; when using an object URI, the storage layer must provide immutability/versioning and hash verification.
- Criteria: unique `(snapshot_id, criterion_key)` and `(snapshot_id, display_order)`; parent criterion must be in the same snapshot and cannot reference itself.
- Alternatives: unique `(snapshot_id, alternative_key)` and display order.
- Scales: unique `(snapshot_id, scale_key)`; scale values unique by `(scale_id, ordinal)` and optionally `(scale_id, stable_value_key)`.
- Matrix: unique `(snapshot_id, alternative_id, criterion_id)`; both referenced rows must belong to that snapshot. A ready snapshot must have the required matrix coverage.

5.3 Session and group constraints

- `sessions.public_slug` unique when non-null. Generate a random slug; do not expose sequential IDs.
- `active_configuration_version_id` must belong to the same session.
- Schedule check: `closes_at IS NULL OR opens_at IS NULL OR closes_at > opens_at`.
- State/timestamp checks: `open` requires `opened_at`; `closed` requires `closed_at`; `archived` requires `archived_at`.
- Only draft/scheduled sessions may change active calculation configuration. Enforce this in the domain service and recheck inside the transaction.
- Configuration versions: unique `(session_id, version_number)` and `(session_id, config_hash)`.
- Groups: unique `(configuration_version_id, group_key)`, unique display order, `allocation >= 0`, and at least one active group with positive allocation.
- Allocation totals should equal an exact denominator such as 10,000 basis points, or be normalized at run time from positive `Numeric` allocations. Recommended: store `allocation_units` as a positive integer and derive normalized power, avoiding fragile floating-point sum checks.

- Question definitions: unique (`configuration_version_id`, `question_key`) and display order. Criterion pairs must have different members and use canonical ordering so the same pair cannot appear twice.
- A session cannot open until snapshot, algorithms, scales, questions, group allocations, and required access controls pass a readiness check. Store readiness results/audit event.

5.4 Invitation, code, and participant constraints

- Invitation token digests are globally unique; plaintext tokens are never stored.
- An invitation may be redeemed once: `redeemed_at` and `redeemed_participant_id` are set atomically.
- `expires_at` > `created_at`; revoked invitations cannot be redeemed.
- Email/identity binding should use both encrypted contact data and a keyed lookup hash if searching/deduplication is required.
- Access-code digests are unique per active scope; codes have an explicit hashing algorithm and parameters. Store only Argon2id/scrypt/bcrypt-style password hashes, never fast unsalted hashes.
- Participant: unique (`session_id`, `participant_id`) and, when a user account exists, unique (`session_id`, `user_id`).
- Invitation-backed participant: unique `invitation_id` when non-null.
- Alias uniqueness is needed only if aliases are publicly displayed; null aliases remain allowed.
- Participant group and configuration must belong to the same session.
- Index participant dashboards by (`session_id`, `access_status`), (`session_id`, `group_id`), and (`session_id`, `completed_at`).
- Access grants use unique token digests, `expires_at`, optional `revoked_at`, and rotation/replacement links.
- Access attempts index (`rate_limit_key_hash`, `attempted_at`) and (`session_id`, `attempted_at`); delete or aggregate them according to the security retention policy.

5.5 Submission and validation constraints

- Unique (`participant_id`, `attempt_number`).
- A replacement's `previous_submission_id` belongs to the same participant and has the immediately preceding attempt number.
- At most one draft and one effective submitted attempt per participant. Use PostgreSQL/SQLite partial unique indexes where supported and enforce the same rule transactionally in the domain service.
- Submitted/superseded attempts require `submitted_at`, `submitted_by`, `answers_hash`, and finalized answer manifest.
- Draft attempts must not have submission metadata.
- `submitted_at` >= `started_at`; other timestamps cannot precede creation.
- Submission's group/configuration/snapshot must match the session and participant at the time of submission.
- Answer: unique (`submission_id`, `question_definition_id`). The question belongs to the submission's configuration version.
- Pairwise answers must identify the question-defined left/right criteria and use an allowed scale value or validated numeric ratio.

- Direct rankings require a complete permutation or an explicit tie policy. Direct ratings require values from the configured scale/range.
- Validation uniqueness should include `submission_id`, `answers_hash`, `configuration_hash`, `validator_implementation_id`, `validator_version`, and `validator_parameter_hash`.
- A `valid` or `valid_with_warnings` weighting validation must have a complete participant weight vector whose nonnegative crisp weights sum to one within a documented tolerance. Fuzzy triples enforce `lower <= middle <= upper`.
- Index validations by `(submission_id, completed_at DESC)` and `(status, completed_at)`.

5.6 Processing, analysis, and report constraints

- Unique `(session_id, run_number)` for processing runs and `(processing_run_id, run_number)` for analysis runs.
- A processing run may begin only for a closed session unless an explicit preview-run flag is set. Preview outputs can never be published as final.
- A succeeded run requires `input_set_hash`, `configuration_hash`, `environment_hash`, `started_at`, `finished_at`, and a `structured_result` artifact.
- Run-submission: unique `(processing_run_id, submission_id)`. Included rows require a valid selected validation; excluded rows require a reason code and safe detail.
- Freeze participant/group allocation in the run-submission row so later access-status changes do not rewrite history.
- Group weights unique by `(processing_run_id, group_id, criterion_id)`.
- Aggregate weights unique by `(processing_run_id, criterion_id)`.
- Rankings unique by `(processing_run_id, scope_type, scope_id, alternative_id)`. Rank is positive; scores must be finite; ties use a documented tie strategy.
- Artifact unique `(owner_type, owner_id, artifact_type, schema_version, sequence)` and stores a content hash.
- Analysis subjects must be authorized for the primary processing run's session.
- Report audience requires scope: participant audience requires `participant_id`; stakeholder audience requires `group_id`; moderator/public have no participant scope.
- Publication requires an approved report or explicitly approved structured result. Unique `(session_id, public_slug, publication_version)`.

5.7 Audit constraints

- Audit events are insert-only to the application database role. No cascade deletion.
- Required fields: event ID, occurred-at UTC, actor type, action, primary entity type/key, session ID when applicable, request/correlation ID, and event hash.
- `before_json` is null for create; `after_json` is null for hard redaction/retirement actions; ordinary update includes a JSON Patch or safe before/after projection.
- Include `reason` for moderator exclusions, group allocation changes, access revocation, manual close/reopen, result approval, redaction, or withdrawal.
- Optional chain: `event_hash = SHA256(canonical_event_without_hash || previous_event_hash)`. Partition chains by session to avoid global write contention.
- A hash chain detects changes only if checkpoints are anchored outside the same administrator's control. Periodically sign/export `audit_chain_checkpoints` to immutable storage for stronger resistance to tampering.

6. Public, private, invitation-only, and access-code behavior

6.1 Authorization decision order

For every participant entry or submission request, evaluate all of these in one application service:

1. The session is in a state that allows entry/submission and the UTC schedule allows it.
2. The session is discoverable only if listing is requested; knowing an unlisted URL is not itself authorization.
3. Enrollment mode permits self-enrollment or a valid invitation claim is present.
4. The required access code, if any, has been verified and rate limits have not been exceeded.
5. The participant/access grant is active, unexpired, unrevoked, and belongs to the session.
6. The participant's group is active and selection/assignment follows the configured mode.
7. Submission limits and resubmission rules allow the requested operation.

6.2 Public open session

- **listed + open + none**: anyone may view and self-enroll.
- If **stakeholder_selection_mode=self_select**, the participant selects among active groups before starting. Store the choice and prevent changes after answers begin.
- Issue a participant access grant in an HttpOnly, Secure, SameSite cookie or equivalent server-side session mechanism.
- Apply bot/rate controls and decide whether anonymous duplicate participation is acceptable. Browser cookies alone do not enforce one-human-one-response.

6.3 Public session with shared access code

- **listed + open + shared_session_code** permits discovery but requires the code before enrollment or before starting a response, as defined by policy.
- Prefer checking the code before creating a participant row to avoid junk enrollments.
- Shared codes provide low-assurance gatekeeping, not individual identity or one-person-one-vote assurance.

6.4 Unlisted open session

- **unlisted + open** is accessible to anyone with the URL, with optional shared code.
- This is useful for tests or low-risk outreach but must be labeled accurately: the URL is a discovery secret, not a durable access control.

6.5 Private invitation-only session

- **unlisted + invitation_only** is the recommended private preset.
- Generate a cryptographically random, high-entropy one-time token. Store only its digest.
- Optionally bind the invitation to an encrypted email/contact hash and preassign its group.
- Redemption occurs transactionally: verify token, expiry, revocation, optional code, and identity binding; create/reuse the participant; mark the invitation redeemed; issue a separate participant access grant.
- Reopening a link after redemption uses the participant access grant, not the invitation token. A moderator may revoke either independently.

- When `per_invitation_code` is enabled, distribute the code through a different channel if it is intended as a second factor.

6.6 Submission and anonymity implications

Support three identity policies explicitly:

- **identified**: identity visible to authorized moderators;
- **pseudonymous**: identity may be held separately, while analysis uses an alias;
- **anonymous**: no identifying data is intentionally collected.

Do not promise anonymity if IP addresses, email delivery logs, or reusable tokens can readily identify a participant. Define retention and access for security metadata. AI inputs should use pseudonymous identifiers and the minimum necessary comment text.

7. End-to-end workflow and auditability

7.1 Import and snapshot a scenario

1. Load and validate every scenario document and referenced file.
2. Reject path traversal, undeclared files used by preprocessing, nonfinite numeric data, unknown criteria/alternatives, and unsafe custom code.
3. Execute approved preprocessing in a controlled import environment.
4. Materialize and validate the decision matrix.
5. Canonicalize documents and matrix data; hash every component and the root manifest.
6. Insert the snapshot, files, criteria, alternatives, scales, matrix, provenance, import tool version, and validation outcome in one transaction.
7. Mark it **ready** only after all completeness checks pass; emit an audit event.

If the original directory is later deleted, the snapshot still contains the evidence and calculation inputs required for reproduction.

7.2 Configure and open a session

1. Create a draft session against one ready snapshot.
2. Create an immutable configuration version containing response rules, group allocations, algorithm configs, validation thresholds, resubmission policy, missing-group policy, and result-visibility policy.
3. Generate deterministic response question definitions from the configuration and snapshot.
4. Configure discoverability, invitation/code behavior, schedule, and moderator roles.
5. Run a readiness validation and store/audit its result.
6. Atomically activate the configuration and transition the session to scheduled/open.

Opening freezes calculation-relevant configuration. Any exceptional override must create a version and an audit event explaining compatibility and impact.

7.3 Enroll a participant

1. Apply the access flow from section 6.
2. Create one session-scoped participant and optional isolated identity record.
3. Bind the participant to one active stakeholder group.

4. Mark invitation redemption and issue a revocable participant access grant.
5. Audit enrollment using pseudonymous identifiers; never log secrets.

7.4 Draft and submit answers

1. Create attempt 1 in **draft** state, pinned to the configuration/snapshot/group.
2. Persist one answer row per question. Draft autosaves may update rows, but meaningful state changes and finalization are audited; avoid logging sensitive full answers on every keystroke.
3. For pairwise format, use canonical criterion-pair questions and the configured preference scale. Require every necessary pair unless incomplete submission is explicitly permitted for validation.
4. For direct format, store whether the target is a criterion or an alternative and whether the participant is rating or ranking. Never infer this from payload shape.
5. At submit, validate payload shape, build the canonical answer manifest, calculate **answers_hash**, set submission metadata, and make the attempt immutable.
6. If resubmission is allowed, create attempt N+1 and atomically mark the old effective submission **superseded** only when the new attempt is successfully submitted.

7.5 Validate a submission

1. Create a validation row with exact submission, answer hash, configuration hash, validator implementation/version, scale, thresholds, and parameter hash.
2. Check completeness, allowed targets/values, pairwise reciprocity and matrix coverage, direct-ranking validity, and any consistency threshold.
3. Store structured messages rather than only a Boolean/error string.
4. Normalize answers into versioned normalized-answer rows.
5. Generate participant-level criterion weights when the response semantics support it; persist crisp/fuzzy values and sum/shape diagnostics.
6. Finish as **valid**, **valid_with_warnings**, **invalid**, or **error** and audit the outcome.

Invalid, error, superseded, and withdrawn submissions cannot be included in a final processing run.

7.6 Close and process a session

1. Close the session automatically at cutoff or explicitly by an authorized moderator. Record effective cutoff and reason.
2. Take a transactionally consistent candidate roster: one effective submitted attempt per eligible participant as of the cutoff.
3. Select a specific validation for every candidate. Record all included and excluded submissions with reasons.
4. Verify minimum submission and required-group policies. Apply the configured missing-group policy explicitly.
5. Compute participant-to-group aggregation, then allocation-weighted cross-group weights, then alternative rankings using the frozen scenario matrix.
6. Store exact run algorithm versions/configuration, application commit/build, Python/package environment, seed if relevant, start/end times, input-set hash, and execution host/job metadata.
7. Persist structured group weights, aggregate weights, rankings, intermediate artifacts, and a schema-versioned clean result object.

8. Mark the run succeeded only after all outputs and hashes commit atomically; otherwise retain a failed run and sanitized failure artifact.

Creating another run never overwrites the first. A moderator chooses which succeeded run is authoritative/publishable through an approval/publication record.

7.7 Analyze and explain

1. Create an analysis run against an immutable processing run.
2. Pin analysis method/version, parameters, comparison runs/subjects, code/environment metadata, and random seed.
3. Store cases and results for sensitivity, robustness, group divergence, participant impact, and comments as structured artifacts.
4. Build an AI-ready **analysis_bundle** containing only approved, minimized, pseudonymized data and explicit provenance links.
5. For AI generation, store provider/model identifier, model configuration, prompt/template ID and hash, input artifact IDs/hashes, timestamps, safety/redaction results, and output hash.
6. Generate separately scoped outputs for moderators, stakeholder groups, and participants. Require human approval before public or consequential use.
7. Publish by creating a versioned publication record. Withdrawal hides it but preserves the audit lineage.

AI text is explanatory output, not a source of calculation truth. Every quantitative claim in a report should carry a machine-readable reference to a structured result/artifact field.

7.8 Audit throughout

Audit events should capture meaningful business transitions, including scenario import, readiness checks, configuration activation, allocation changes, session opening/closing, invitation issue/revocation/redemption, participant disable/withdrawal, submission finalization/supersession/withdrawal, validation outcomes, run inclusion/exclusion, processing/analysis completion, report approval, publication, and redaction.

Use one correlation ID for every application transaction so the event, row changes, and background jobs can be traced together. Hashes provide content identity; audit events provide who/when/why; immutable run rosters provide exact computational lineage. All three are required for reproducibility.

8. Missing requirements, risks, and decisions to resolve

The schema can support the following options, but product policy must settle them before implementation.

8.1 Highest-priority decisions

1. **What does “direct” input target?** The request says participants rank or score options, while the current scenario template describes direct linguistic ratings of criteria and the weighting pipeline expects criterion weights. Direct alternative scores do not naturally produce AHP criterion weights for TOPSIS. Recommended resolution: support **direct_rating** of criteria for weighting first; model direct alternative rating/ranking as a distinct response target and analysis/ranking path, not as an interchangeable AHP input.

2. **How should within-group preferences aggregate?** Arithmetic mean, geometric mean, aggregation of pairwise judgments, aggregation of priority vectors, and fuzzy operators can produce different outcomes. Make this a versioned algorithm choice. For AHP pairwise judgments, choose deliberately between aggregation of individual judgments and aggregation of individual priorities.
3. **What happens when a group has no valid submissions?** Recommended default is **fail** and moderator resolution. If allocations are renormalized, the report and audit must disclose it.
4. **What does a group allocation mean?** Confirm whether it is fixed normative voting power, a default that moderators may change, or derived from an external mandate. Changes after opening should be prohibited.
5. **When is a session configuration frozen?** Recommended: at open, with no calculation-relevant changes after the first submission.
6. **What is the authoritative final result?** Multiple successful runs may exist. Define approval roles and whether publication points to one approved run/version.

8.2 Participation and identity decisions

- Is one-human-one-submission required, or only one browser/invitation/user-account submission?
- Can a public participant change stakeholder group? Recommended: only before the first answer is saved.
- Can moderators reassign a participant, and how should prior submissions be treated?
- Are participants allowed to withdraw data after processing/publication, and does withdrawal cause a new run rather than rewriting old results?
- What is “completed”: submitted, valid, or included in a run? Recommended fields/timestamps should distinguish all three.
- Are participant names, emails, demographics, or only aliases collected? Define consent, retention, export, and deletion obligations.
- Who may see raw comments and individual impact analysis? These can reveal identity even in nominally anonymous sessions.
- How many invitation sends, redemptions, draft attempts, and resubmissions are allowed?

8.3 Method and data decisions

- Define scale semantics separately for pairwise AHP, fuzzy AHP, and direct rating; a generic five/seven-point label is insufficient.
- Define AHP consistency thresholds and whether warning-level submissions remain usable.
- Define fuzzy number shape and serialization (recommended triangular triples initially).
- Define missing values, tied direct rankings, incomplete answers, duplicate alternatives, and nonfinite matrix values.
- Define hierarchical criteria support. A parent-child criterion tree changes required pairwise questions and weight roll-up rules.
- Decide whether group-specific alternative performance matrices are supported or there is one objective snapshot matrix.
- Define numerical precision, rounding only for display, tie tolerance, deterministic sorting, and random seeds.
- Confirm PyDecision adapter input/output conventions during implementation without encoding those library-specific shapes into core tables.

8.4 Reproducibility and operational risks

- The current snapshot stores configuration JSON but not referenced data bytes or `functions.py`; deletion or edits would prevent full reproduction. Snapshot files and the materialized matrix are required.
- Declared scenario versions can be reused accidentally. Content hashes, not version labels, must define identity.
- Custom preprocessing code is a supply-chain/security risk. Execute it only during controlled import with an allowlist/sandbox strategy; retain its source and environment as evidence.
- SQLite does not provide PostgreSQL-like concurrent write behavior and has limited timezone/type enforcement. It is appropriate for development, not production load/security testing.
- Partial unique indexes are supported by SQLite and PostgreSQL but still need transactional service checks to give consistent errors and handle races.
- A same-database hash chain is tamper-evident only against unsophisticated changes. External signed checkpoints/backups are needed for stronger claims.
- Generic JSON artifacts can become undocumented data dumps. Require schemas, versions, hashes, size limits, and retention rules.
- AI reports can hallucinate, leak sensitive comments, or overstate causality. Minimize inputs, cite structured artifact paths, review outputs, and never let AI mutate calculation results.

8.5 Governance questions

- Who can create, open, pause, close, rerun, exclude submissions, approve results, publish, and withdraw reports?
- Which moderator actions require a reason or dual approval?
- What retention period applies to PII, access logs, raw answers, comments, artifacts, and audit records?
- What must be exportable for independent reproduction, and in what open format?
- Should participants see their validation/consistency diagnostics before final submission, and could that bias responses?
- Are published aggregate results subject to minimum group-size suppression to prevent re-identification?

9. Final recommended schema outline

This outline names implementation-level fields without prescribing SQLAlchemy code. All foreign-key ID fields use the same portable UUID storage type. All timestamps are UTC. JSON fields include a schema version where their shape is not intrinsically fixed.

9.1 Identity and roles

users

- `user_id` PK
- `identity_provider`, `identity_subject`; unique together
- `display_name`, `email_ciphertext`, `email_lookup_hash` nullable
- `status`
- audit timestamps

session_role_assignments

- `session_role_assignment_id` PK

- `session_id` FK -> sessions, `user_id` FK -> users
- `role` (owner, moderator, analyst, auditor, report_approver)
- `granted_at`, `granted_by`, `revoked_at`, `revoked_by`
- unique active (`session_id`, `user_id`, `role`)

9.2 Scenario snapshot

scenario_definitions

- `scenario_definition_id` PK
- `scenario_key` unique, `title`, `domain`, `description`
- `status`, audit timestamps

scenario_snapshots

- `scenario_snapshot_id` PK
- `scenario_definition_id` FK
- `declared_version`, `scenario_type`, `schema_version`, `status`
- `title`, `domain`, `summary`, `policy_question`
- `manifest_json`, `manifest_schema_version`, `root_hash` unique
- `materialized_input_hash`
- `source_uri` nullable, `importer_version`, `import_environment_json`
- `created_at`, `created_by`, `ready_at`

scenario_snapshot_files

- `scenario_snapshot_file_id` PK
- `scenario_snapshot_id` FK
- `logical_path`, `file_role`, `media_type`, `byte_size`, `content_hash`
- `inline_bytes` nullable, `immutable_object_uri` nullable
- exactly one content location; unique snapshot/path

scenario_criteria

- `criterion_id` PK, `scenario_snapshot_id` FK
- `criterion_key`, `name`, `description`, `direction`, `data_type`, `unit`
- `parent_criterion_id` nullable self-FK, `required`, `display_order`
- `source_column`, `metadata_json`

scenario_alternatives

- `alternative_id` PK, `scenario_snapshot_id` FK
- `alternative_key`, `name`, `description`, `display_order`, `metadata_json`

scenario_scales

- `scale_id` PK, `scenario_snapshot_id` FK
- `scale_key`, `name`, `scale_type`, `ordered`, `definition_version`, `metadata_json`

scenario_scale_values

- `scale_value_id` PK, `scale_id` FK

- `stable_value_key`, `label`, `ordinal`
- `numeric_value` Numeric nullable
- `fuzzy_lower`, `fuzzy_middle`, `fuzzy_upper` Numeric nullable
- `metadata_json`

scenario_matrix_values

- composite PK or unique (`scenario_snapshot_id`, `alternative_id`, `criterion_id`)
- `value_numeric` Numeric nullable, `value_json` nullable
- `source_provenance_json`

9.3 Session configuration

sessions

- `session_id` PK, `scenario_snapshot_id` FK
- `public_slug` unique, `title`, `description`, `admin_notes`
- `status`, `discoverability`, `enrollment_mode`, `access_code_mode`
- `identity_policy`, `stakeholder_selection_mode`
- `opens_at`, `closes_at`, `opened_at`, `paused_at`, `closed_at`, `cancelled_at`, `archived_at`
- `active_configuration_version_id` nullable same-session FK
- `created_at/by`, `updated_at/by`

session_configuration_versions

- `configuration_version_id` PK, `session_id` FK, `version_number`
- `scenario_snapshot_id` FK
- `response_format`, `response_target_type`, `scale_id` FK
- `allow_resubmissions`, `max_submissions_per_participant`
- `allow_incomplete_submission`, `minimum_valid_submissions`
- `missing_group_policy`, `required_group_policy_json`
- `consistency_threshold` nullable
- `configuration_json`, `schema_version`, `config_hash`
- `created_at/by`, `activated_at/by`

session_stakeholder_groups

- `session_stakeholder_group_id` PK
- `configuration_version_id` FK
- `group_key`, `name`, `description`
- `allocation_units` nonnegative integer, `display_order`, `is_active`
- `within_group_algorithm_config_id` nullable FK
- `created_at/by`

algorithm_implementations

- `algorithm_implementation_id` PK
- `stable_key`, `role`, `conceptual_method`
- `provider`, `library_name`, `library_version`, `implementation_version`
- `adapter_version`, `parameter_schema_json`, `capabilities_json`

- `status`, `created_at`
- unique stable key/provider/library/implementation version

session_algorithm_configs

- `session_algorithm_config_id` PK
- `configuration_version_id` FK, `algorithm_implementation_id` FK
- `role`, `execution_order`
- `parameter_json`, `parameter_schema_version`, `parameter_hash`
- unique configuration/role/order

response_question_definitions

- `question_definition_id` PK, `configuration_version_id` FK
- `question_key`, `question_type`, `display_order`, `required`
- `criterion_id` nullable, `left_criterion_id` nullable, `right_criterion_id` nullable
- `alternative_id` nullable, `scale_id` nullable
- `prompt_snapshot`, `metadata_json`
- checks enforcing the correct target columns for each question type

9.4 Access and participation

session_invitations

- `invitation_id` PK, `session_id` FK
- `configuration_version_id` FK, `assigned_group_id` nullable FK
- `token_digest` unique, `token_hint` nullable
- `identity_lookup_hash` nullable, `identity_ciphertext` nullable
- `status`, `expires_at`, `max_redemptions` default 1
- `sent_at`, `send_count`, `redeemed_at`, `redeemed_participant_id` nullable
- `revoked_at/by/reason`, `created_at/by`

session_access_codes

- `access_code_id` PK, `session_id` FK, `invitation_id` nullable FK
- `scope`, `code_hash`, `hash_algorithm`, `hash_parameters_json`
- `valid_from`, `expires_at`, `max_uses`, `use_count`
- `revoked_at/by`, `created_at/by`

access_attempts

- `access_attempt_id` PK, `session_id` FK
- `invitation_id`, `access_code_id` nullable
- `attempted_at`, `outcome`, `reason_code`
- `rate_limit_key_hash`, `network_metadata_json` minimized

participants

- `participant_id` PK, `session_id` FK
- `configuration_version_id` FK, `session_stakeholder_group_id` FK
- `user_id` nullable FK, `invitation_id` nullable unique FK

- `alias` nullable, `access_status`
- `enrolled_at`, `joined_at`, `started_at`, `submitted_at`, `completed_at`
- `disabled_at/by/reason`, `withdrawn_at/by/reason`
- `created_at/by`, `updated_at/by`

participant_identities

- `participant_id` PK/FK
- `display_name_ciphertext`, `email_ciphertext`, `email_lookup_hash`, other approved PII
- `consent_version`, `consented_at`, `retention_until`, `redacted_at/by`

participant_access_grants

- `access_grant_id` PK, `participant_id` FK
- `token_digest` unique, `issued_at`, `expires_at`, `last_used_at`
- `revoked_at/by/reason`, `replaced_by_grant_id` nullable self-FK

9.5 Submission and validation

submissions

- `submission_id` PK
- `session_id`, `participant_id`, `configuration_version_id`, `scenario_snapshot_id`, `session_stakeholder_group_id` FKs with composite integrity
- `attempt_number`, `previous_submission_id` nullable self-FK
- `status`, `response_format`, `response_target_type`
- `answer_manifest_json`, `answer_schema_version`, `answers_hash` nullable until submit
- `started_at`, `last_saved_at`, `submitted_at/by`, `superseded_at/by`, `withdrawn_at/by/reason`
- `client_metadata_json` minimized, `created_at/by`, `updated_at/by`

submission_answers

- `submission_answer_id` PK, `submission_id` FK, `question_definition_id` FK
- `raw_value_json`, `value_schema_version`, `raw_value_hash`
- optional typed convenience columns: `selected_scale_value_id`, `numeric_value`, `rank_value`
- `answered_at`, `response_time_ms` nullable
- unique submission/question

submission_comments

- `submission_comment_id` PK, `submission_id` FK
- optional `criterion_id`, `alternative_id`
- `comment_text`, `audience`, `created_at`
- `moderation_status`, `redacted_text`, `redacted_at/by/reason`

submission_validations

- `validation_id` PK, `submission_id` FK
- `answers_hash`, `configuration_hash`
- `validator_implementation_id` FK, `validator_version`, `parameter_json/hash`

- `status`, `completion_ratio`, `consistency_ratio` nullable, `quality_metrics_json`
- `started_at`, `completed_at`, `validated_by_actor_type/id`
- `input_hash`, `output_hash`, `failure_code/detail` nullable

validation_messages

- `validation_message_id` PK, `validation_id` FK
- `severity`, `code`, `question_definition_id` nullable
- `safe_message`, `parameters_json`, `display_order`

validation_normalized_answers

- `unique (validation_id, submission_answer_id)`
- `normalized_value_json`, optional crisp/fuzzy typed columns, `normalizer_version`

participant_criterion_weights

- `unique (validation_id, criterion_id)`
- `crisp_weight` nullable
- `fuzzy_lower`, `fuzzy_middle`, `fuzzy_upper` nullable
- `derivation_metadata_json`

9.6 Processing output

processing_runs

- `processing_run_id` PK, `session_id` FK, `run_number`
- `configuration_version_id`, `scenario_snapshot_id` FKs
- `status`, `is_preview`, `requested_at/by`, `started_at`, `finished_at`
- `cutoff_at`, `missing_group_policy`
- `input_set_hash`, `configuration_hash`, `scenario_root_hash`
- `application_version`, `source_revision`, `python_version`
- `environment_json/hash`, `random_seed` nullable
- `worker/job_id` nullable, `failure_code/detail` nullable

processing_run_algorithms

- `processing_run_algorithm_id` PK, `processing_run_id` FK
- `role`, `execution_order`, `algorithm_implementation_id` FK
- frozen provider/library/implementation/adapters versions
- frozen `parameter_json`, schema version, and hash

processing_run_submissions

- `processing_run_submission_id` PK, `processing_run_id` FK
- `submission_id` FK, `validation_id` nullable FK
- `participant_id`, `session_stakeholder_group_id`
- `inclusion_status`, `reason_code`, `reason_detail`
- frozen `group_allocation_units`, optional `within_group_participant_weight`
- `submission_hash`, `validation_output_hash`

group_criterion_weights

- unique (processing_run_id, session_stakeholder_group_id, criterion_id)
- crisp/fuzzy weight columns
- contributing_submission_count, derivation_artifact_id

aggregate_criterion_weights

- unique (processing_run_id, criterion_id)
- crisp/fuzzy weight columns
- derivation_artifact_id

alternative_rankings

- alternative_ranking_id PK, processing_run_id FK, alternative_id FK
- scope_type (aggregate, stakeholder_group), scope_group_id nullable
- score_numeric nullable, fuzzy score JSON/columns nullable
- rank, tie_group nullable, positive/negative ideal distances nullable
- derivation_artifact_id

run_artifacts

- run_artifact_id PK, processing_run_id FK
- artifact_type, name, schema_version, sequence
- content_json nullable, inline_bytes nullable, immutable_object_uri nullable
- media_type, byte_size, content_hash
- created_at, producer_metadata_json

9.7 Analysis and reports

analysis_runs

- analysis_run_id PK, processing_run_id FK, run_number
- analysis_type, status, method_key/version, parameters_json/hash
- input_artifact_manifest_json/hash, random_seed nullable
- environment/source revision metadata
- requested/started/finished/failure metadata

analysis_run_subjects

- analysis_run_subject_id PK, analysis_run_id FK
- subject_type, exactly one authorized subject ID/key
- role (included, baseline, comparison, excluded), reason

analysis_comparison_runs

- unique (analysis_run_id, comparison_processing_run_id)
- comparison role/label

analysis_artifacts

- same artifact envelope pattern as run_artifacts

- required **analysis_bundle** artifact for AI/report consumption

report_runs

- **report_run_id** PK, **session_id** FK
- **processing_run_id** FK, **analysis_run_id** nullable FK
- **audience**, **participant_id** nullable, **session_stakeholder_group_id** nullable
- **generation_type** (template, ai), **status**
- **template_id/version/hash**
- AI provider/model/config and prompt hash nullable
- exact input artifact manifest/hash
- requested/started/finished/failure metadata

report_outputs

- **report_output_id** PK, **report_run_id** FK, **version_number**
- **content_format**, **content_text** or immutable artifact reference
- **structured_claims_json**, **content_hash**
- **approval_status**, **reviewed_at/by**, **review_notes**
- **redacted_at/by/reason**, **created_at**

publications

- **publication_id** PK, **session_id** FK
- **report_output_id** nullable FK, **processing_run_id** FK
- **public_slug**, **publication_version**
- **published_at/by**, **withdrawn_at/by/reason**
- **visibility**, **metadata_json**

9.8 Audit

audit_events

- **audit_event_id** PK, sortable by time
- **session_id** nullable, **occurred_at**
- **actor_type**, **actor_id** nullable, **actor_display_snapshot** nullable
- **action**, **entity_type**, **entity_id**
- **reason_code**, **reason_text** nullable
- **before_json**, **after_json**, **patch_json** nullable and secret-filtered
- **request_id**, **correlation_id**, **causation_event_id** nullable
- **source_metadata_json** minimized
- **previous_event_hash** nullable, **event_hash**

audit_event_links

- unique (**audit_event_id**, **linked_entity_type**, **linked_entity_id**, **relationship**)

audit_chain_checkpoints

- **checkpoint_id** PK, **session_id** nullable
- **through_event_id**, **chain_hash**, **signed_hash**, **signing_key_id**

- `external_anchor_uri` nullable, `created_at`

9.9 Recommended changes from the current repository model

The existing models can evolve toward this design rather than being discarded:

- Replace the composite (`scenario_id`, `scenario_version`) identity with a surrogate `scenario_snapshot_id` plus content hashes; keep the declared values as metadata/alternate keys.
- Extend snapshot storage to include source file bytes, custom-function evidence, and the canonical matrix.
- Split session `visibility` into discoverability and enrollment mode; replace `require_access_code/access_code_type` with explicit access-code rows.
- Move method choices into immutable session algorithm configurations and freeze them again on runs.
- Add immutable session configuration versions and make groups belong to a version.
- Add missing group description/order and store allocation in exact units. Do not persist `normalized_voting_power` as mutable source-of-truth data; derive or freeze it in a run.
- Add invitation, access-grant, and optional separated participant-identity entities.
- Pin submissions directly to session, snapshot, configuration, and group for historical integrity.
- Replace answer-only JSON with question definitions plus answer rows and retain a canonical manifest/hash at submission.
- Replace `is_valid` and `errors_json` with validation status, structured messages, normalized answers, and weight rows.
- Add processing/analysis/report/publication entities rather than treating `processed` and `published` as terminal session states.
- Implement the currently empty audit domain with append-only structured events.

This design preserves SQLite development portability while providing the lineage, constraints, security boundaries, and concurrency model needed for a PostgreSQL production deployment.